## Step 1-3: Install/Import Libraries and Download NLTK Resources

```
#MOHANASUNDARAM
#241801169
import nltk
import pandas as pd
import numpy as np
import string
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Download necessary NLTK resources
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
```

## Step 4-5: Load and Prepare the Dataset

We will use a publicly hosted CSV subset of the Amazon Fine Food Reviews dataset for immediate execution without needing Kaggle credentials.

```
# Load Amazon Fine Food Reviews dataset (using a public mirror source to avoid Kaggle auth i
url = "https://raw.githubusercontent.com/mohitgupta-omg/Kaggle-SMS-Spam-Collection-Dataset-/
# Since fine foods dataset is large, we can download a smaller sample or mock dataset that s
# or download a 10k slice directly from a public URL:
csv_url = "https://media.githubusercontent.com/media/juliangruber/amazon-fine-food-reviews-s

try:
    # Reading first 10,000 rows directly
    df = pd.read_csv(csv_url, nrows=10000)
    print("Successfully loaded 10,000 reviews from the dataset.")
except Exception as e:
    print("Error loading from GitHub, generating a synthetic dataset for demonstration:", e)
    # Fallback synthetic dataset
    data = {
        'Text': [
            "This is a great product! The shipping was fast and it arrived in perfect condit
            "Very disappointed with the taste. It was stale and tasted awful.",
            "Good value for money. My kids love these healthy snacks!",
            "Terrible customer service and the product was damaged during shipping.",
            "Excellent quality, highly recommend to everyone looking for organic tea.",
            "I am disappointed because it has too much sugar, not healthy at all.",
            "Great taste, fast delivery, and premium packaging!",
            "Not worth the price. Extremely disappointed with this purchase."
        ] * 1250  # Repeat to match user scale
    }
    df = pd.DataFrame(data)
```

```
# Clean missing values
df = df.dropna(subset=['Text'])
reviews_df = df[['Text']].head(10000).copy()
display(reviews_df.head())
```

## Step 6-7: Text Preprocessing Function

We convert text to lowercase, remove punctuation, tokenize, and filter out stopwords using NLTK.

```
# Load English stopwords
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    if not isinstance(text, str):
        return ""
    # a. Convert to lowercase
    text = text.lower()
    # b. Remove punctuation and special characters
    text = text.translate(str.maketrans('', '', string.punctuation))
    # c. Tokenize
    tokens = word_tokenize(text)
    # d. Remove stopwords
    filtered_tokens = [word for word in tokens if word not in stop_words]
    # e. Join back into a string
    return " ".join(filtered_tokens)

# Apply preprocessing to all reviews
reviews_df['Cleaned_Text'] = reviews_df['Text'].apply(preprocess_text)
display(reviews_df[['Text', 'Cleaned_Text']].head())
```

## Step 8: Convert Text to Vectors using TF-IDF

```
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(reviews_df['Cleaned_Text'])
print(f"TF-IDF Matrix shape: {tfidf_matrix.shape}")
```

## Step 9-11: Retrieval Function and Query Examples

```
def retrieve_reviews(query, k=3):
    # a. Preprocess query
    cleaned_query = preprocess_text(query)
    # b. Vectorize query
    query_vector = vectorizer.transform([cleaned_query])
    # c. Compute Cosine Similarity
    similarity_scores = cosine_similarity(query_vector, tfidf_matrix).flatten()
    # d. Identify top k matching indices
    top_indices = similarity_scores.argsort()[-k:][::-1]

    print(f"\nQuery: \"{query}\"")
    print("="*50)
```

```
        for rank, idx in enumerate(top_indices, 1):
            score = similarity_scores[idx]
            if score > 0:
                print(f"Rank {rank} (Score: {score:.4f})")
                print(f"Original: {reviews_df.iloc[idx]['Text']}")
                print(f"Cleaned:  {reviews_df.iloc[idx]['Cleaned_Text']}")
                print("-"*50)
            else:
                print(f"Rank {rank}: No relevant match found.")
                print("-"*50)

    # Run queries
    retrieve_reviews("disappointed", k=5)
    retrieve_reviews("great quality", k=3)
```

```
    ---------------------------------------------------------------------------
    NameError                                 Traceback (most recent call last)
    /tmp/ipykernel_969/856971276.py in <cell line: 0>()
         23
         24 # Run queries
    ---> 25 retrieve_reviews("disappointed", k=5)
         26 retrieve_reviews("great quality", k=3)

    /tmp/ipykernel_969/856971276.py in retrieve_reviews(query, k)
          1 def retrieve_reviews(query, k=3):
          2     # a. Preprocess query
    ----> 3     cleaned_query = preprocess_text(query)
          4     # b. Vectorize query
          5     query_vector = vectorizer.transform([cleaned_query])

    NameError: name 'preprocess_text' is not defined
```

Next steps: Explain error